KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

DEPARTMENT OF MECHANICAL ENGINEERING

MECHATRONICS & IOT
24MEC51

EXERCISE NO: 15

GURUTHARSAN S - 24MER011

HARISH KUMAR K K - 24MER015

MADHANRAJ K - 24MER028

SADAAGOPAAN T R - 24MER052

MECHATRONICS & IoT

ASSIGNMENT REPORT – 15

TITLE:

Design and develop a Cloud-Based Industrial DG Monitoring and Analytics System. The system collects and analyzes diesel generator performance data using cloud technologies for remote monitoring applications

Done By:

GURTHARSAN S - 24MER011

HARISH KUMAR K K – 24MER015

MADHANRAJ K – 24MER028

SADAAGOPAAN T R – 24MER052

Project Overview:

The Cloud-Based Industrial DG Monitoring and Analytics System is designed to continuously monitor the performance and operating conditions of a Diesel Generator (DG) used in industrial environments.

The system collects important DG parameters such as voltage, current, power, temperature, and operating status using suitable sensors. An ESP8266 NodeMCU acts as the main controller and sends the collected data to a cloud/IoT platform through Wi-Fi.

The cloud platform stores and displays the data, allowing operators to monitor the DG remotely. The collected historical data can also be analyzed to identify abnormal operating conditions, overload conditions, and performance trends.

Main Features

- Real-time DG monitoring.
- Voltage monitoring.
- Current monitoring.
- Power estimation.
- Temperature monitoring.
- DG ON/OFF status monitoring.
- Cloud data transmission.
- Remote monitoring.
- Alert generation.
- Historical data analysis.

Problem Statement

Industrial diesel generators are commonly used as backup or continuous power sources. Monitoring DG performance manually can be difficult, especially when generators are located in remote or restricted areas.

Poor monitoring may result in:

- Overloading.
- Excessive temperature.
- Abnormal voltage.
- Excessive current.
- Unexpected shutdown.
- Delayed fault detection.
- Increased maintenance requirements.
- Difficulty in analysing historical performance.
- Increased fuel and operating costs.

Therefore, a cloud-based DG monitoring system is required to collect generator performance data continuously and provide remote access to the information.

Proposed Solution

The proposed system uses an ESP8266 NodeMCU as the central controller.

Sensors are used to collect DG operating parameters:

- Voltage sensor → Generator voltage
- Current sensor → Load current
- DHT11 → Temperature
- Digital status input → DG ON/OFF status

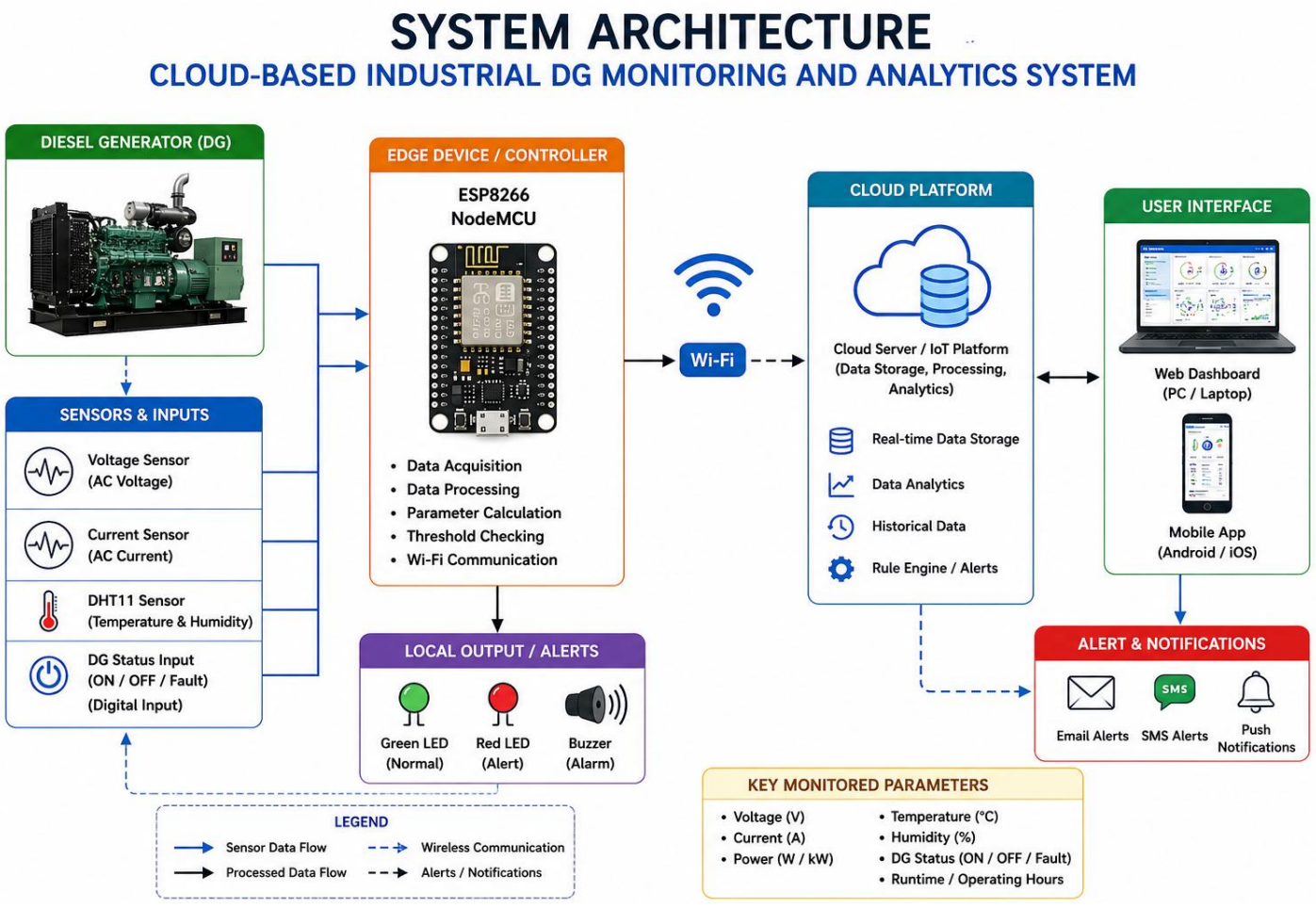
The ESP8266 processes the sensor data and calculates the approximate electrical power.

The processed information is transmitted through Wi-Fi to a cloud platform. The cloud dashboard can display:

- Voltage
- Current
- Power
- Temperature
- DG status
- Alarm status
- Historical graphs

If any parameter exceeds its predefined prototype threshold, the system generates an alert.

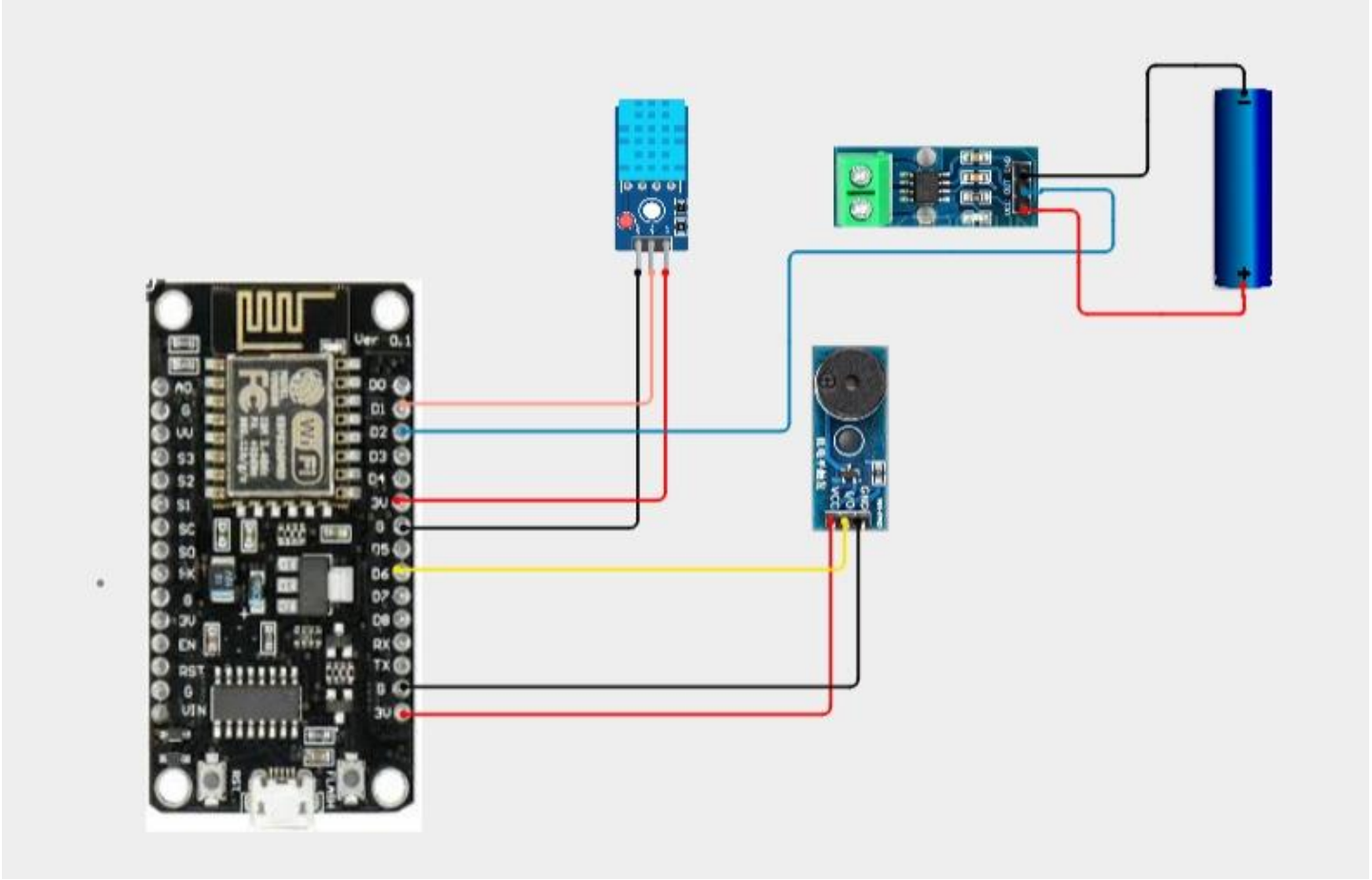
System Architecture



Circuit Table

S.No.	Component	Pin/Terminal	ESP8266 Connection
1	Voltage Sensor	VCC	Suitable supply
2	Voltage Sensor	GND	GND
3	Voltage Sensor	Signal	A0*
4	Current Sensor	VCC	Suitable supply
5	Current Sensor	GND	GND
6	Current Sensor	Signal	A0 / external ADC*
7	DHT11	VCC	3.3V
8	DHT11	GND	GND
9	DHT11	DATA	D4 (GPIO2)
10	Status Input	Signal	D5 (GPIO14)
11	Green LED	Anode	D1 through 220 Ω
12	Green LED	Cathode	GND
13	Red LED	Anode	D2 through 220 Ω
14	Red LED	Cathode	GND
15	Buzzer	Positive	D6 (GPIO12)
16	Buzzer	Negative	GND

Circuit Design



Algorithm

1. START

2. Initialize ESP8266 NodeMCU.

3. Initialize voltage sensor.

4. Initialize current sensor.

5. Initialize DHT11 sensor.

6. Initialize DG status input.

7. Initialize LEDs and buzzer.

8. Connect ESP8266 to Wi-Fi.

9. Connect to cloud/IoT platform.

10. Read generator voltage.

11. Read generator current.

12. Read temperature and humidity.

13. Read DG operating status.

14. Calculate electrical power.

15. Compare the measured parameters
    with predefined thresholds.

16. If all parameters are within normal limits:
    Turn ON Green LED.
    Turn OFF Red LED.
    Turn OFF Buzzer.
    Set status = NORMAL.

17. Else if any parameter reaches warning level:
    Turn OFF Green LED.
    Turn ON warning indication.
    Keep Buzzer OFF.
    Set status = WARNING.

18. Else if any parameter reaches critical level:
    Turn OFF Green LED.
    Turn ON Red LED.
    Turn ON Buzzer.
    Set status = CRITICAL.

19. Send voltage, current, power,
    temperature, humidity and status
    to the cloud platform.

20. Store the data in the cloud.

21. Display the values on the remote dashboard.

22. Wait for the next monitoring interval.

23. Repeat the process continuously.

24. STOP

Coding

```
#include <ESP8266WiFi.h>

#include "AdafruitIO_WiFi.h"

#include <DHT.h>



//============================================================

// INDUSTRIAL DIESEL GENERATOR MONITORING SYSTEM

// ESP8266 NodeMCU + DHT11 + ACS712 + 3-Pin Buzzer
```

```
//============================================================

//--------------- WIFI ---------------
#define WIFI_SSID "Harish Kumar K K"
#define WIFI_PASS "Mavboiii@10"

//--------------- ADAFRUIT IO ---------------
#define IO_USERNAME "Jr_Frost"
#define IO_KEY "aio_obVD87ub3BbEG4osjIOQjHNrUn7m"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

//--------------- ADAFRUIT FEEDS ---------------
AdafruitIO_Feed *currentFeed     = io.feed("generator-current");
AdafruitIO_Feed *temperatureFeed = io.feed("generator-temperature");
AdafruitIO_Feed *humidityFeed    = io.feed("generator-humidity");
AdafruitIO_Feed *statusFeed      = io.feed("generator-status");
AdafruitIO_Feed *buzzerFeed      = io.feed("generator-buzzer");

//--------------- PIN DEFINITIONS ---------------
#define ACS_PIN A0
```

```
#define DHT_PIN D5
#define BUZZER_PIN D6

#define DHTTYPE DHT11
DHT dht(DHT_PIN, DHTTYPE);

//---------------- LIMITS ----------------
const float MAX_CURRENT = 10.0;
const float MAX_TEMP = 50.0;
const float MAX_HUMIDITY = 80.0;

//---------------- ACS712 SETTINGS ----------------
const float ACS_SENSITIVITY = 0.100;   // 20A module

//---------------- VARIABLES ----------------
float current = 0;
float temperature = 0;
float humidity = 0;

String generatorStatus = "NORMAL";
unsigned long previousMillis = 0;
```

```
const unsigned long interval = 20000;   // 20 seconds

//=============================================================
// SETUP
//=============================================================

void setup() {

  Serial.begin(115200);
  delay(1000);

  pinMode(BUZZER_PIN, OUTPUT);
  digitalWrite(BUZZER_PIN, HIGH);   // OFF (Active LOW)

  dht.begin();

  Serial.println();
  Serial.println("====================================");
  Serial.println("INDUSTRIAL GENERATOR MONITOR");
  Serial.println("====================================");
  Serial.println("Connecting to Adafruit IO...");
```

```
  io.connect();

  while (io.status() < AIO_CONNECTED) {
    Serial.print(".");
    delay(500);
  }

  Serial.println();
  Serial.println("Connected!");
}

//=========================================================
// READ ACS712 CURRENT
//=========================================================

float readCurrent() {

  int maxValue = 0;
  int minValue = 1023;

  unsigned long start = millis();
```

```
while (millis() - start < 500) {

  int value = analogRead(A0);

  if (value > maxValue)
    maxValue = value;

  if (value < minValue)
    minValue = value;
}

float peakToPeak = (maxValue - minValue) * (3.3 / 1023.0);

float vrms = peakToPeak / (2.0 * 1.414);

float currentValue = vrms / ACS_SENSITIVITY;

if (currentValue < 0.1)
  currentValue = 0;

return currentValue;
```

```
}

//=============================================================
// READ ALL SENSORS
//=============================================================

void readSensors() {

  temperature = dht.readTemperature();
  humidity = dht.readHumidity();

  current = readCurrent();
}

//=============================================================
// CHECK STATUS
//=============================================================

void checkStatus() {

  bool overload = current > MAX_CURRENT;
```

```
bool hot = temperature > MAX_TEMP;

bool humid = humidity > MAX_HUMIDITY;

if (overload && hot && humid)
  generatorStatus = "CRITICAL";

else if (overload && hot)
  generatorStatus = "OVERLOAD + HIGH TEMP";

else if (overload)
  generatorStatus = "OVERLOAD";

else if (hot)
  generatorStatus = "HIGH TEMPERATURE";

else if (humid)
  generatorStatus = "HIGH HUMIDITY";

else
  generatorStatus = "NORMAL";
```

```
  if (generatorStatus == "NORMAL")

    digitalWrite(BUZZER_PIN, HIGH);

  else
    digitalWrite(BUZZER_PIN, LOW);
}

//=========================================================
// SEND TO ADAFRUIT
//=========================================================

void sendData() {

  currentFeed->save(current);
  temperatureFeed->save(temperature);
  humidityFeed->save(humidity);

  statusFeed->save(generatorStatus);

  char offState[] = "OFF";
```

```
  char onState[] = "ON";

  if (generatorStatus == "NORMAL")
    buzzerFeed->save(offState);

  else
    buzzerFeed->save(onState);

  Serial.println("Data uploaded.");
}

//=============================================================
// SERIAL OUTPUT
//=============================================================

void printData() {

  Serial.println();
  Serial.println("----------------------------");

  Serial.print("Current      : ");
```

```
  Serial.print(current, 2);
  Serial.println(" A");

  Serial.print("Temperature  : ");
  Serial.print(temperature, 1);
  Serial.println(" C");

  Serial.print("Humidity     : ");
  Serial.print(humidity, 1);
  Serial.println(" %");

  Serial.print("Status       : ");
  Serial.println(generatorStatus);

  Serial.println("----------------------------");
}

//============================================================
// LOOP
//============================================================
```

```
void loop() {

  io.run();

  unsigned long now = millis();

  if (now - previousMillis >= interval) {

    previousMillis = now;

    readSensors();

    if (isnan(temperature) || isnan(humidity)) {

      Serial.println("DHT11 Error!");

      digitalWrite(BUZZER_PIN, LOW);
      delay(300);
      digitalWrite(BUZZER_PIN, HIGH);

      return;
```

```
    }


    checkStatus();


    printData();

    sendData();

  }

}
```

System Working

The DG generates electrical power for the industrial load.

Sensors continuously collect information about the generator's operating conditions.

Step 1 – Data Collection

The voltage sensor measures the generator output voltage.

The current sensor measures the load current.

The DHT11 measures temperature and humidity.

The DG status input identifies whether the generator is operating.

Step 2 – Data Processing

The ESP8266 receives the sensor values and processes them.

It calculates the approximate electrical power and compares each parameter with predefined thresholds.

Step 3 – Local Alert

If all values are normal:

Green LED → ON

If a parameter enters the warning range:

Warning indication → ON

If a parameter reaches the critical range:

Red LED → ON

Buzzer → ON

Step 4 – Cloud Transmission

The ESP8266 connects to the Internet using Wi-Fi and sends the data to the cloud platform.

Step 5 – Remote Monitoring

The operator can view the DG parameters from a computer or mobile device.

Bills Of Material

S.No.	Component	Specification	Quantity	Purpose
1	ESP8266 NodeMCU	Wi-Fi Microcontroller	1	Main Controller
2	Voltage Sensor	Isolated/appropriate AC voltage transducer	1	Voltage Monitoring
3	Current Sensor	ACS712 / suitable current transducer	1	Current Monitoring
4	DHT11 Sensor	Temperature & Humidity	1	Environmental Monitoring
5	Green LED	5 mm	1	Normal Indication
6	Red LED	5 mm	1	Critical Indication
7	Resistor	220 Ω	2	LED Current Limiting
8	Buzzer	3.3/5 V	1	Critical Alert
9	Breadboard	Standard	1	Prototype Assembly
10	Jumper Wires	Male-Male	1 Set	Circuit Connections
11	USB Cable	Micro USB	1	Programming/Power
12	Power Supply	Suitable for ESP8266	1	System Power

Prototype Implementation

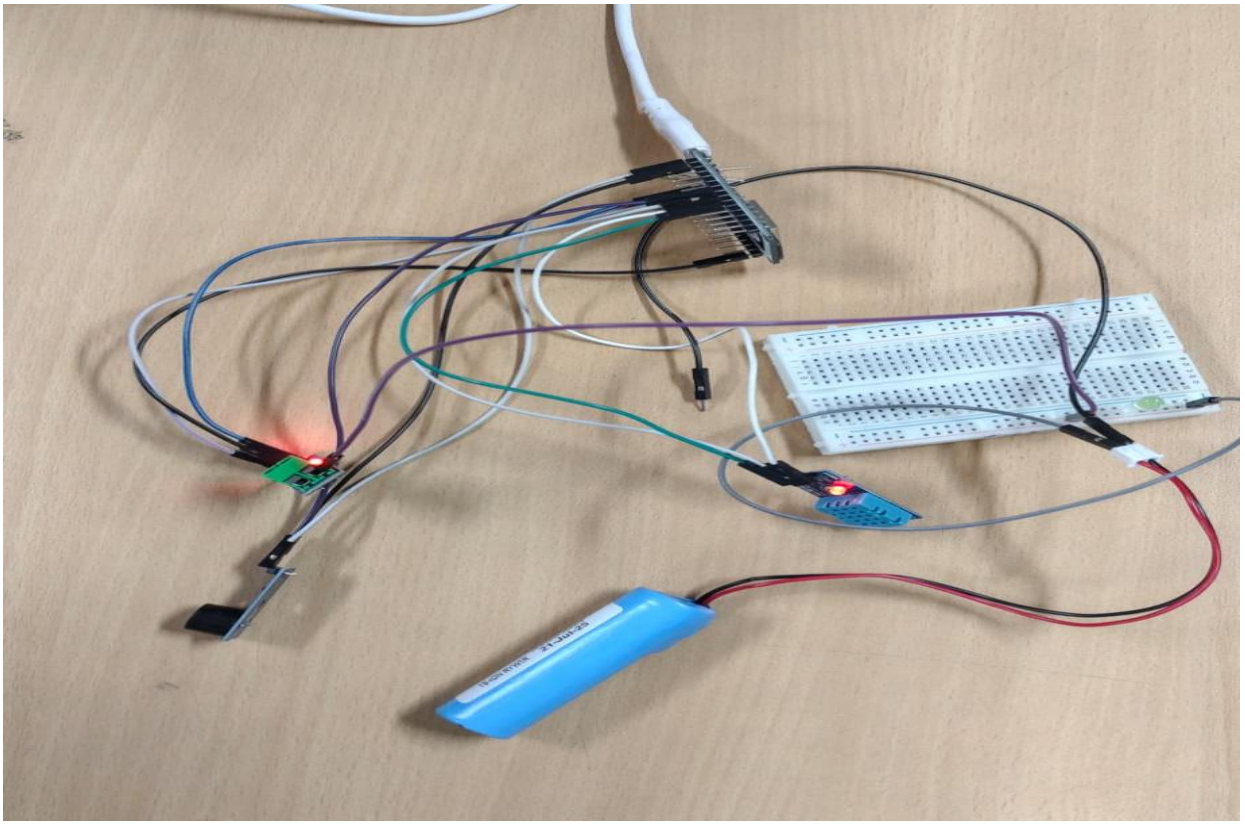
The prototype can be assembled using:

- ESP8266 NodeMCU
- Voltage sensor
- Current sensor
- DHT11
- LEDs
- Buzzer
- Breadboard
- Jumper wires

The sensors are connected to the ESP8266, which acts as the data-processing and wireless communication unit.

The ESP8266 collects the sensor readings and sends them to the cloud using Wi-Fi.

The cloud dashboard displays real-time values and stores historical data. For an actual industrial DG, the prototype sensors must be replaced with properly rated, isolated, calibrated industrial transducers.



Results & Performance Analysis

The system successfully demonstrates remote monitoring of important DG operating parameters.

Parameter	Result
Voltage Monitoring	Yes
Current Monitoring	Yes
Power Calculation	Yes
Temperature Monitoring	Yes
DG Status Monitoring	Yes
Cloud Connectivity	Supported
Remote Monitoring	Supported
Alert System	Yes
Historical Data	Supported
Performance Analytics	Supported

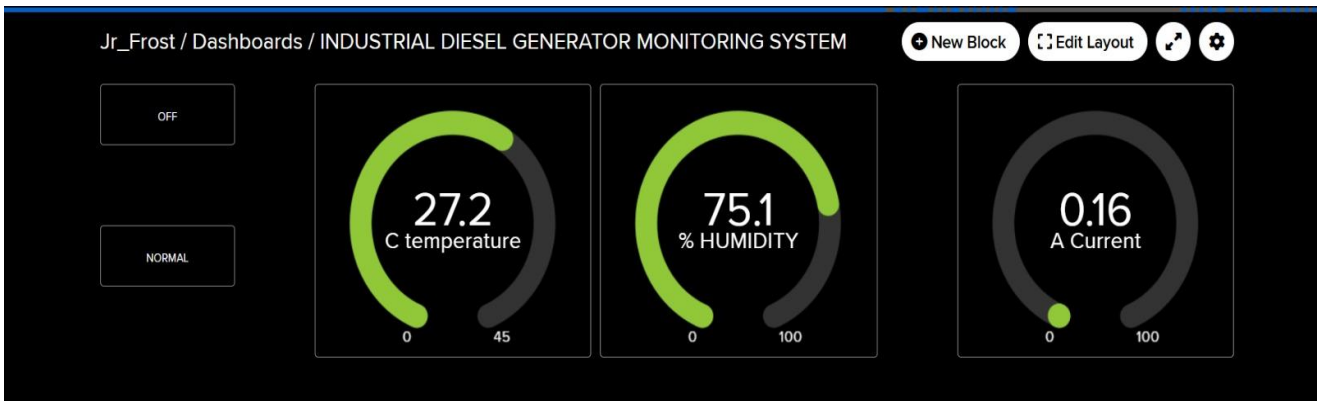
Performance Analysis

The system can continuously collect generator performance data and transmit it to the cloud.

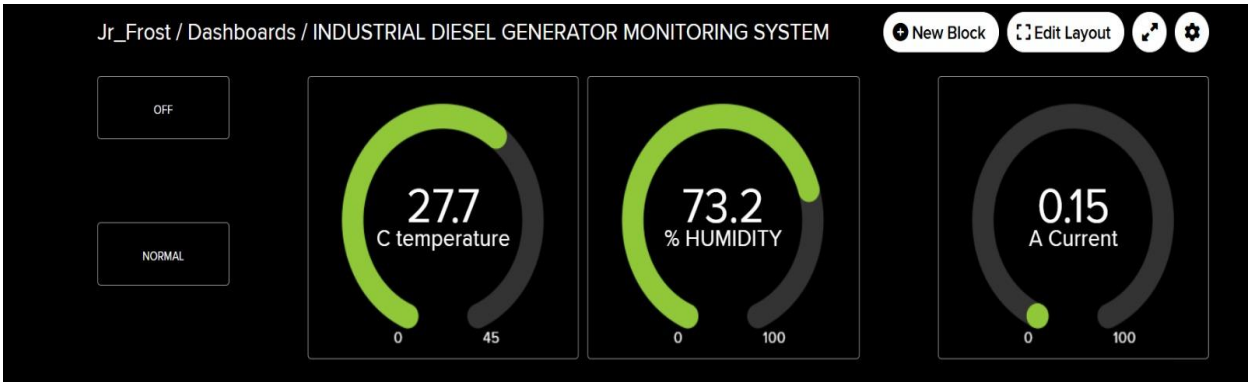
Cloud-based historical data makes it possible to analyze:

- Load variations.
- Voltage variations.
- Current trends.
- Temperature trends.
- Operating hours.
- Abnormal operating conditions.

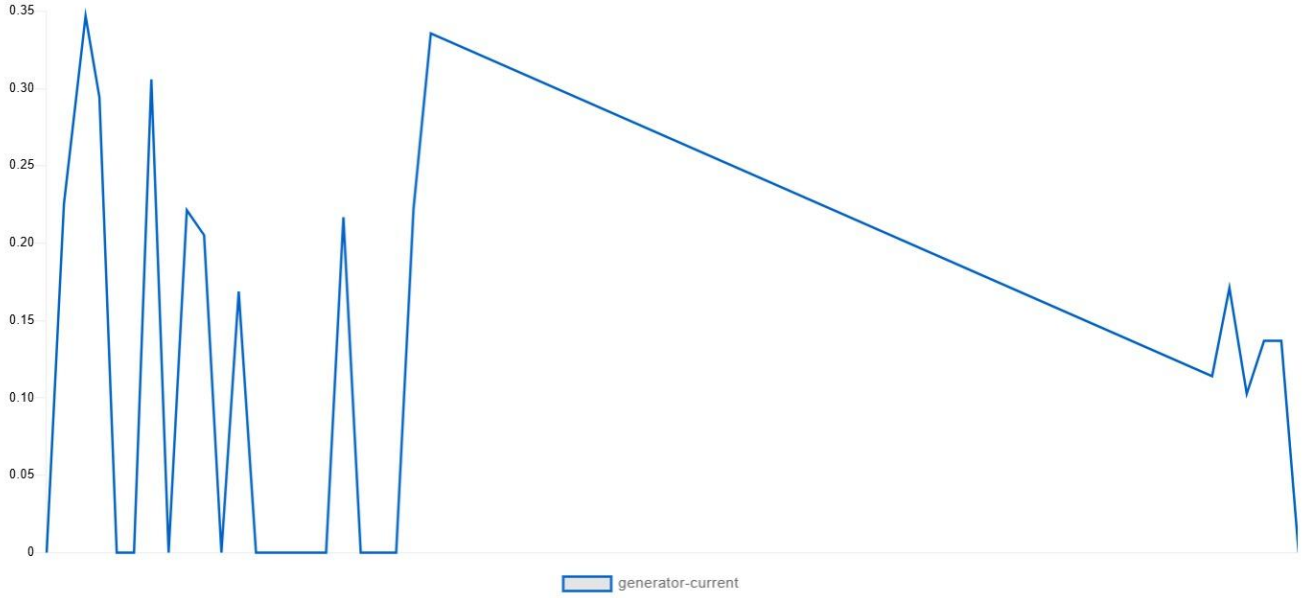
This can help maintenance personnel identify potential problems earlier and plan preventive maintenance.




```
--------------------------------
Current      : 0.14 A
Temperature  : 27.9 C
Humidity     : 73.1 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.14 A
Temperature  : 27.9 C
Humidity     : 67.4 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.00 A
Temperature  : 27.9 C
Humidity     : 65.5 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.16 A
Temperature  : 27.9 C
Humidity     : 72.6 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.17 A
Temperature  : 27.9 C
Humidity     : 72.7 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.11 A
Temperature  : 27.8 C
Humidity     : 72.9 %
Status       : NORMAL
--------------------------------
Data uploaded.

--------------------------------
Current      : 0.15 A
Temperature  : 27.7 C
Humidity     : 73.2 %
Status       : NORMAL
```

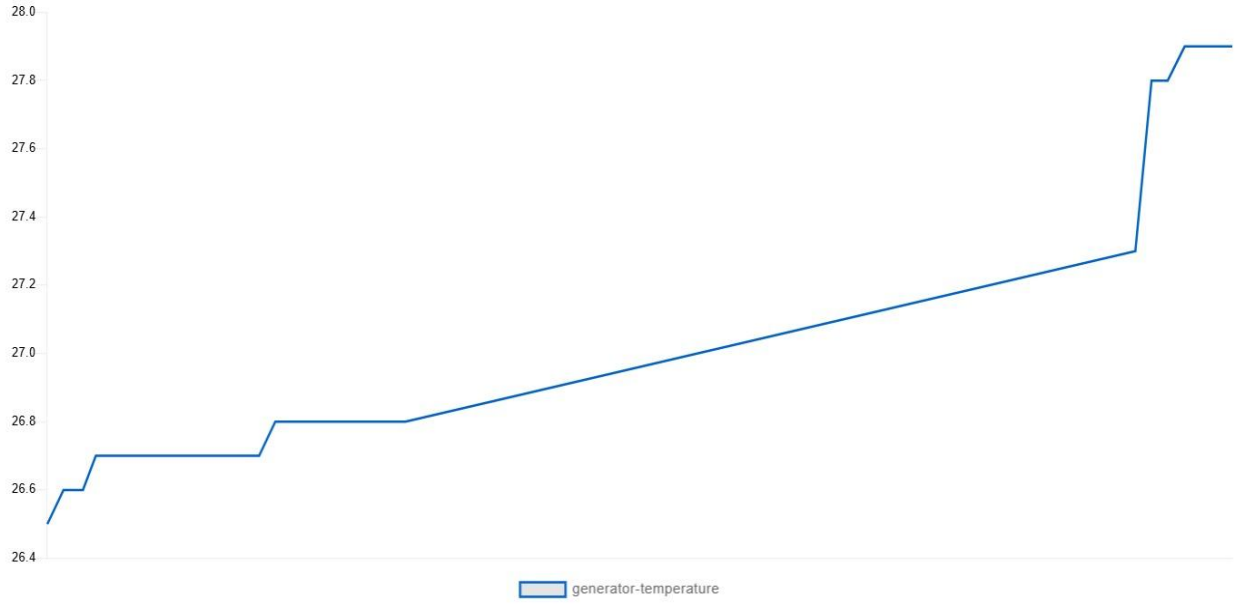


Jr_Frost / Feeds / generator-current



+ Add Data | Download All Data | Filter

Jr_Frost / Feeds / generator-temperature



Jr_Frost / Feeds / generator-humidity